

LAST COMMAND

AI 활용 기술서

개발에 사용한 AI 도구, 대표 프롬프트, 단계별 활용 내역과 검증 절차를 기록한 공모전 제출 문서

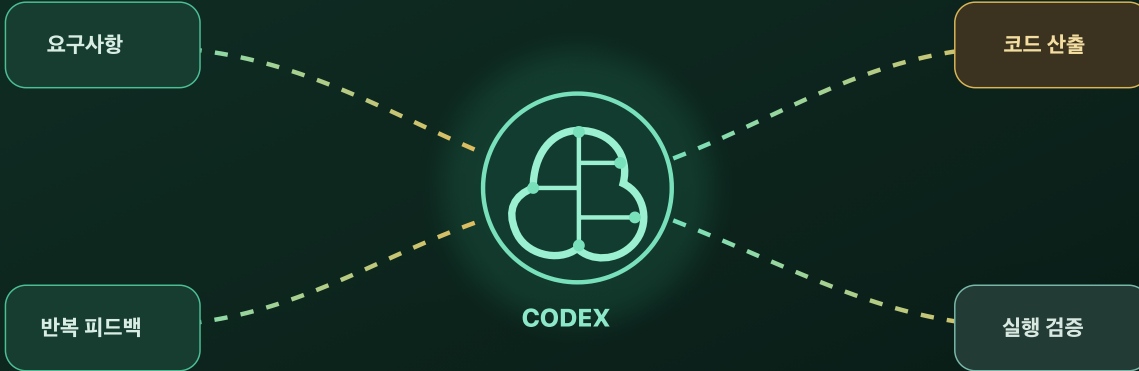
AI-ASSISTED DEVELOPMENT

PROMPT LOG

VERIFICATION

RUNTIME AI

HUMAN-IN-THE-LOOP DEVELOPMENT



PROMPT → PATCH → TEST → REVIEW

1개 AI 도구

OpenAI Codex 중심 활용

7개 개발 단계

초기 구현부터 문서화까지

사람 최종 승인

요구·수정·커밋·배포 결정

AI 도구와 역할

본 프로젝트는 **OpenAI Codex**를 개발 보조 에이전트로 사용했습니다. 사용자가 목표와 제약, 수정 우선순위를 결정하고 AI가 저장소 분석·코드 작성·리팩터링·검증 스크립트 작성·문서화를 수행한 뒤, 실제 실행 결과를 기준으로 반복 수정했습니다.



Human-in-the-loop 공동 개발

AI의 출력은 바로 제출 결과로 간주하지 않았습니다. 기능 단위로 브라우저 실행, 콘솔 오류, 저장 상태와 버튼 동작을 확인한 후 사용자가 커밋·푸시를 명시적으로 승인했습니다.

PRIMARY AI TOOL

OpenAI Codex

GPT-5 기반 개발 에이전트. 자연어 요구를 파일 단위 구현 계획과 코드 패치로 변환하고, 기존 구조를 읽어 변경 범위를 제한했습니다.

코드 생성

리팩터링

VERIFICATION TOOL

Headless Chrome

AI 도구가 아니라 검증 도구입니다. 실제 Phaser 장면을 실행해 승리·패배·버튼·저장값·콘솔 오류를 자동 확인했습니다.

브라우저 테스트

ENGINEERING TOOL

Git · Shell

검색, 문법 검사, 변경 비교, 커밋 이력 기록에 사용했습니다. 각 기능은 독립 커밋으로 남겨 AI 활용 결과를 추적할 수 있습니다.

변경 추적

재현

Codex 활용 범위

- ✓ 현재 프로젝트 디렉터리와 기존 코드 구조 분석
- ✓ Phaser 장면·엔티티·AI·전투 시스템 분리 설계
- ✓ 게임 코드 작성, 버그 원인 추적과 최소 범위 수정
- ✓ 공간 해시 기반 성능 개선과 주석·README 작성
- ✓ 브라우저 자동 검증 시나리오 작성 및 결과 판독
- ✓ 제출용 게임 소개서와 AI 활용 기술서 구성

사용하지 않은 항목

- 실행 중 외부 LLM/API 호출
- 이미지 생성 AI 및 외부 이미지 에셋
- 음악·음성 생성 AI
- 외부 상용 게임 코드·그래픽 복제
- 사용자 개인정보 수집용 백엔드

사람이 결정한 것

게임 방향, 필수 기능, 오리지널리티 조건, 수정 요청, 우선순위, 최종 결과 채택, Git 커밋 및 원격 푸시 여부.

AI가 지원한 것

구현안 제안, 모듈화, 파일 생성·수정, 오류 진단, 반복 테스트, 코드 품질 점검, 제출 문서 초안과 시각 구성.

프롬프트 설계와 실제 예시

프롬프트는 한 번의 “게임 제작” 요청에 그치지 않고, 플랫폼 제약과 합격 기준을 포함한 기준 프롬프트를 먼저 제공한 뒤 실제 플레이 결과에 따라 짧은 수정 프롬프트를 반복하는 방식으로 운영했습니다.

CONTEXT

게임 장르, 공모전 목적, 기존 저장소

OBJECTIVE

이번 단계에서 완성할 기능과 사용자 경험

CONSTRAINTS

정적 배포, 원본성, 성능, 파일 구조

VERIFICATION

실행·조작·오류·저장 여부 확인 항목

MASTER PROMPT · 초기 제작

“퍼스트퀀4에서 영감을 받은 대규모 동료 부대 실시간 전투 RPG를 만들어주세요. 단, 캐릭터·이름·스토리·맵·그래픽·음악·UI·고유 시스템을 복제하지 말고 완전히 새로운 오리지널 게임으로 제작해야 합니다. HTML5, JavaScript, Phaser 3, GitHub Pages에서 별도 서버 없이 실행... 아군 AI 상태는 FOLLOW, ATTACK, RETREAT, PROTECT, REGROUP, DEFEND로 분리...”

활용 의도: 장르 목표뿐 아니라 저작권 경계, 기술 스택, 파일 구조, AI 입력·상태, 성능, 검증 체크리스트를 동시에 고정

반복 개선 프롬프트

COMBAT UX

““적이라 겹쳐지지 않게 하고 자동공격 되게 해줘. 스페이스 눌러서 공격하는 게 아니라.””

결과: 적·아군 충돌 및 separation 보장, 자동 표적 획득과 쿨타임 공격으로 변경.

BUG FIX

““패배하거나 승리했을 때 다시 시작, 메인 화면 버튼이 왜 안 눌러지지?””

결과: 카메라 이동 후 입력 좌표가 어긋나는 원인을 찾고 오버레이 자식의 scroll factor를 고정.

CAMPAIGN

““지도를 만들어서 이동하면서 최종 맵 클리어 하는 건 어떨까? 만들어줘.””

결과: 4지역 전술 지도, 스테이지 해금, 생존 부대 계승, localStorage 저장 구현.

SCORING

““살아남은 유닛이나 체력, 클리어 시간 등으로 라운드 종료되고 점수 부여되게 하면 좋을 듯.””

결과: 순수 계산 모듈, S~D 등급, 항목별 점수, 지역 최고·캠페인 누적 점수 구현.

프롬프트 → 산출물 변환 원칙

1. 먼저 읽기

수정 전 파일과 Git 상태를 확인해 기존 사용자 환경을 보존.

2. 역할 분리

AI, 전투, 캠페인, 점수를 별도 모듈로 유지해 한 파일 집중 방지.

3. 실행으로 판정

설명이나 코드 모양이 아니라 실제 Chrome 동작과 저장값으로 완료 여부 판단.

※ 위 문장은 실제 개발 대화의 대표 요청을 의미가 변하지 않는 범위에서 발췌·정리했습니다. 전체 초기 프롬프트에는 조작, 전투 수치, 맵, AI 성향, UI, README와 검증 조건이 더 상세하게 포함되었습니다.

단계별 AI 활용 내역

각 요구는 “저장소 확인 → 구현 → 문법·브라우저 검증 → 변경 비교 → 사용자 승인 → 커밋” 순서로 처리했습니다. 아래 Git 커밋은 실제 저장소에서 확인 가능한 결과 근거입니다.

d53661a	초기 게임 전체 구현 15개 파일, 1,817줄 규모의 Phaser 프로젝트 구성. 메뉴, 전장, 유닛, 부대 AI, 전투, 공간 해시, README 생성.	BASE BUILD 15 FILES
be25c9e	자동 공격과 유닛 충돌 개선 Space 수동 공격을 제거하고 근접 자동 공격으로 전환. 적과 유닛이 완전히 겹치지 않도록 물리 충돌·간격 보정.	COMBAT UX 5 FILES
93f09a3	결과 화면 버튼 입력 오류 수정 카메라가 이동한 전장에서 화면 고정 UI의 렌더 좌표와 입력 좌표가 달라지는 원인을 진단하고 자식 객체까지 고정.	BUG FIX HIT AREA
bfaf69f	4스테이지 스토리 캠페인 전술 지도, 해금 순서, 생존 부대 계승, 브라우저 저장, 지역별 설정 데이터를 독립 모듈로 추가.	CAMPAIGN 691 LINES
ac1678f	미션 다양화와 단계형 보스 구출·거점 방어·장치 파괴 목표, 증원 웨이브, 3단계 총지휘관 패턴과 전용 HUD 추가.	MISSION AI BOSS PHASE
cc21297	라운드 점수와 전술 등급 생존, HP, 시간, 구출, 성공 보너스를 계산하는 ScoreSystem과 S~D 결과 UI, 최고 점수 저장 구현.	SCORING PURE MODULE
6e9babf	게임 소개 제출 문서 현재 구현을 기준으로 게임 개요, 플레이, AI, 점수, 실행·배포를 정리한 5페이지 PDF와 편집 원본 제작.	DOCUMENT 5 PAGES

AI가 생성·수정한 주요 산출물

AISystem.js 인식·결정·행동 분리, 4개 성향과 6개 상태	BattleSystem.js 자동 표적, 쿨타임 피해, 지휘관 오라	WorldMapScene.js 스테이지 경로, 선택, 해금, 점수 표시
CampaignSystem.js 부대 계승과 localStorage 진행 저장	ScoreSystem.js Phaser 비의존 결과 점수 순수 계산	README / PDF 실행·배포·AI 설명과 제출 자료

반복 활용 패턴

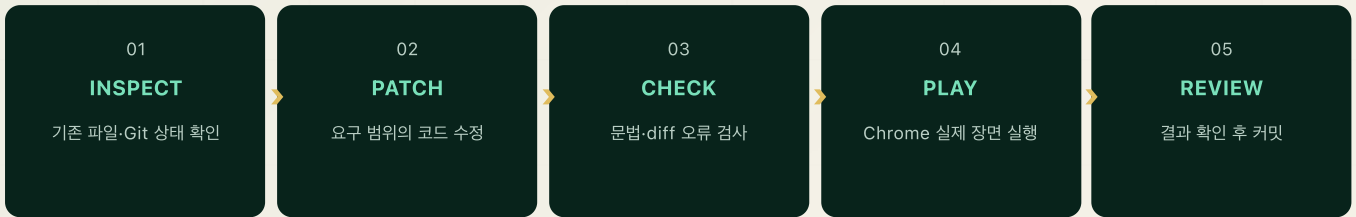
사용자가 실제 플레이에서 불편을 발견 → 짧은 수정 프롬프트 → AI가 관련 코드와 입력 구조 조사 → 최소 패치 → 동일 상황 재검증.

추적 가능성

기능별 독립 커밋과 명확한 메시지를 사용했습니다. 심사자는 저장소 이력에서 각 프롬프트 단계가 어떤 파일 변경으로 이어졌는지 확인할 수 있습니다.

검토와 품질 검증

생성형 AI 출력의 정확성을 보장하기 위해 “코드가 작성되었다”를 완료 조건으로 사용하지 않았습니다. 정적 검사와 실제 Chrome 실행을 함께 사용하고, 실패한 검증은 원인을 확인해 수정 후 재실행했습니다.



검증 영역	방법	확인 내용	상태
JavaScript 문법	<code>node --check</code>	변경된 ES Module 전체 구문 오류 확인	PASS
변경 품질	<code>git diff --check</code>	공백 오류와 잘못된 패치 흔적 확인	PASS
정적 실행	로컬 HTTP 서버	CDN Phaser 및 상대경로 모듈 로드 확인	PASS
게임 시작	Headless Chrome	메뉴 → 지도/Quick Battle → GameScene 진입	PASS
핵심 전투	장면 상태 검사	이동, 자동 공격, 1·2·3 명령, AI 상태 변화	PASS
캠페인 목표	목표별 시나리오	구출, 35초 방어, 장치 파괴, 보스 3단계	PASS
결과 화면	승리·패배 강제 발생	등급·점수 항목, 지도/메인 버튼 hit area	PASS
저장 데이터	localStorage 판독	스테이지 최고 점수와 캠페인 누적 점수	PASS
브라우저 오류	Console/Runtime 수집	검증 시 JavaScript 예외 없음	0 ERROR

15,609

90초·8/10 생존·2명 구출 예시 점수

A

동일 검증 시 전술 등급

0

최종 브라우저 콘솔 오류

실패를 숨기지 않는 검증

테스트 코드도 처음부터 항상 정답은 아니었습니다. 결과 UI가 Phaser Container 내부로 이동해 일반 display list 검색이 실패했을 때, 실제 장면 구조를 조사하고 테스트가 Container 자식을 검사하도록 수정한 뒤 다시 통과시켰습니다.

남아 있는 한계

브라우저 자동 테스트 스크립트는 검증 중 임시로 실행했으며 저장소에는 테스트 프레임워크 형태로 유지하지 않았습니다. 향후 정식 회귀 테스트를 저장소에 포함하면 재현성이 더 높아집니다.

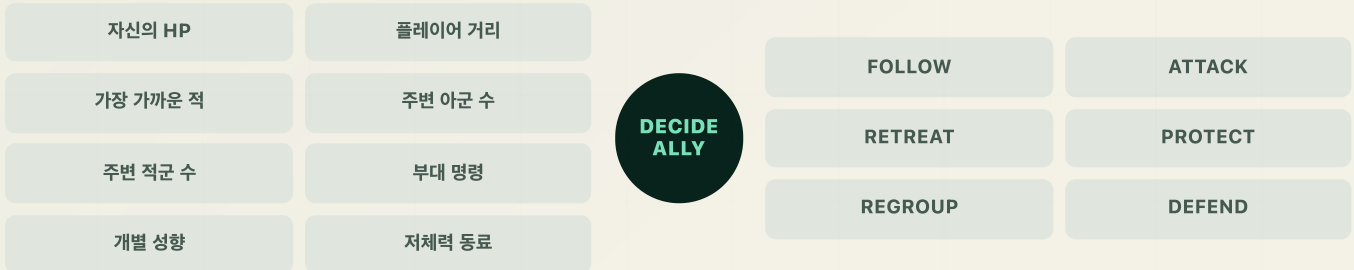
사용자 승인 지점

기능 구현 후 변경 내용과 검증 결과를 사용자에게 보고했으며, 사용자가 “커밋 푸시”를 요청한 단계에서만 Git 커밋과 원격 저장소 반영을 수행했습니다. AI가 임의로 배포 범위를 확장하지 않도록 승인 경계를 유지했습니다.

게임 AI와 책임 있는 활용

개발에 사용한 생성형 AI와 게임 실행 중의 부대 AI는 서로 다릅니다. 게임은 외부 AI 서버에 접속하지 않으며, AISystem.js 의 명시적인 규칙과 상태 전이로 각 병사의 행동을 결정합니다.

런타임 부대 AI 구조



병사별 판단은 200~500ms 간격으로 분산되고, SpatialHash가 주변 셀의 유닛만 검색합니다. 판단 함수가 인식·행동 실행과 분리되어 향후 행동 트리, 강화학습 정책 또는 LLM 어댑터로 교체할 수 있습니다.

개발 생성형 AI

- 도구: OpenAI Codex
- 시점: 제작·수정·검증·문서화 과정
- 입력: 사용자의 자연어 요구와 저장소 코드
- 출력: 코드 패치, 테스트 절차, 문서 초안
- 통제: 사용자 승인 및 실제 실행 검증

게임 런타임 AI

- 도구: 자체 제작 규칙 기반 AISystem
- 시점: 브라우저에서 플레이하는 동안
- 입력: HP, 거리, 병력 수, 명령, 성향
- 출력: 6개 상태와 이동·공격 대상
- 네트워크: 외부 API 호출 없음

위험과 대응

코드 오류·환각	문법 검사만으로 끝내지 않고 실제 Chrome에서 장면과 저장값을 확인했습니다.
요구 범위 이탈	기존 구조와 Git 상태를 먼저 읽고, 사용자 요청에 해당하는 파일만 단계적으로 수정했습니다.
저작권·모방 위험	특정 상용 게임의 캐릭터, 지명, 스토리, UI, 그래픽, 음악을 사용하지 않는 조건을 기준 프롬프트에 명시했습니다.
개인정보·네트워크	게임은 백엔드가 없으며 진행 데이터는 사용자의 브라우저 localStorage에만 저장됩니다.
AI 활용 과정	사용하지 않은 이미지·음악 생성 AI와 런타임 LLM을 명시하고, 규칙 기반 게임 AI와 구분했습니다.

재현 가능한 개발 절차

프롬프트 확인 → 저장소 상태 조사 → 파일 단위 패치 → 정적 검사 → Chrome 시나리오 검증 → 변경 보고 → 사용자 승인 후 커밋·푸시

근거 저장소

GitHub Repository · <https://github.com/dongkyun2331/last-command>

주요 확인 파일: js/systems/AISystem.js, SpatialHash.js, BattleSystem.js, CampaignSystem.js, ScoreSystem.js 및 Git commit history.